🌐 NetworkPolicy in Kubernetes

What is NetworkPolicy?

- 🛡️ A NetworkPolicy is a Kubernetes resource used to `control network traffic between Pods`.
- 👉 Similar to a `firewall for` Pods.
- By default, Kubernetes allows:
  - Pod A ⟶ Pod B ✅
  - Pod B ⟶ Pod C ✅
  - Pod C ⟶ Pod A ✅
- With NetworkPolicy, you can restrict:
  - ➡ Incoming traffic (`Ingress`)
  - ➡ Outgoing traffic (`Egress`)

✅ Amazon VPC CNI

- Amazon VPC CNI supports Kubernetes `NetworkPolicy` in modern EKS versions.
- Modern EKS clusters using `recentAmazon VPC CNI versions` support NetworkPolicies natively.

🕑 Period	🌐 Net wor king Solution	🛡️ Net wor kPolicy Suppor t	💡 Notes
📜 **Earlier Amazon VPC CNI Versions**	AWS VPC CNI	❌ Not Suppor ted	Kubernetes NetworkPolicy resources existed, but `AWS VPCCNIcouldnotenforce them`
🔄 **Calico + Amazon VPC CNI**	AWS VPC CNI + Calico	✅ Supported via Calico	`Calico` enforced NetworkPolicies while `AWS VPC CNI handled networking`
🚀 **Recent Amazon VPC CNI (EKS)**	AWS VPC CNI	✅ Native Suppor t	AWS VPC CNI can now enforce Kubernetes NetworkPolicies `without requiring Calico`

🛡️ Without NetworkPolicy vs With NetworkPolicy

🧩 Feature	❌ Without Net wor kPolicy	✅ With NetworkPolicy
📦 **Pod Communication**	Any Pod can communicate with any Pod	Only explicitly allowed Pods can communicate
🔐 **Security**	Less secure	More secure
🚦 **TrafficControl**	No traffic control	Fine-grained ingress and egress control
⚠️ **Attack Surface**	Higher risk of lateral movement attacks	Limits attack surface
🏗️ **Microservices Isolation**	Difficult to enforce	Easy to enforce
🗄️ **Database Protection**	Any Pod may reach the database	Only authorized Pods can access the database
🏢 **Multi-Tenant Clusters**	Poor isolation	Strong `namespace/workload` isolation
🎯 **Compliance & Governance**	Harder to meet requirements	Easier to enforce security policies
🛡️ **ZeroTrust Model**	Not possible	Suppor ted

🛡️ Kubernetes NetworkPolicy: Ingress vs Egress

🧩 Feature	📥 Ingress Traffic	📤 Egress Traffic
📖 **Meaning**	Traffic coming **into a Pod**	Traffic going **out from a Pod**
🚦 **Direction**	Incoming	Outgoing
🎯 **Controls**	Who can access the Pod	Where the Pod can connect
⚙️ **Policy Type**	`Ingress`	`Egress`
💡 **Use Case**	Allow Frontend → Backend ✅	Allow Backend → Database / Internet ✅
🔓 **Default (No Net wor kPolicy)**	Allowed	Allowed
❓ Who	Ingress controls who can send traffic to a Pod	Egress controls where a Pod can send traffic.

🛡️ Kubernetes NetworkPolicy Scenarios

🎯 Scenario 1: Deny All Traffic to a Namespace (No Pod should send or receive traffic.)

- 🎯 Requirement :
- No Pod in the production namespace should:
  - Receive traffic (Ingress) ❌
  - Send traffic (Egress) ❌

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: production
spec:
  podSelector: {}                    # 👉 Select all Pods in the production namespace.
  policyTypes:
  - Ingress
  - Egress
```

Policy Type	Action
Ingress	🚫 Block all incoming traffic
Egress	🚫 Block all outgoing traffic

Result:

Source = Destination	Status
frontend ——————▶ backend	❌ Blocked
backend ——————▶ mysql	❌ Blocked
frontend ——————▶ Internet	❌ Blocked
DNS Server ——————▶ frontend	❌ Blocked

Why use a Deny-All NetworkPolicy?

- 🛡️ To implement a `Zero TrustSecurityModel`.
- 🚫 First `block all traffic`, then explicitly allow only the required communication between Pods..

✅ Scenario 2: Allow Traffic Only from Frontend to Backend

- 🎯 Requirement :
  - Only the `Frontend Pod` should be able to communicate with the `Backend Pod`.
  - All other Pods must be `denied access.`

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:
    matchLabels:
      app: backend              # ✅ This policy applies only to: Backend Pod

  policyTypes:
  - Ingress                     # ✅ Controls incoming traffic to Backend Pods.

  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend         # ✅ Allow traffic only from Pods with: app: frontend
```

Result Table :

Source Pod	Label	Backend Access
Frontend	app=frontend	✅ Allowed
Database	app=mysql	❌ Denied
Prometheus	app=prometheus	❌ Denied
Any Other Pod	Any Label	❌ Denied

- 👉 This NetworkPolicy selects backend Pods and `allows ingress traffic only` from Pods labeled `app=frontend`; all other Pods are `denied access` to the backend.

✅ Scenario 3: Allow Traffic on Frontend + Port 8080 Specific Port Only

- 🎯 Requirement

- Allow traffic to Backend Pods only on `TCP Port 8080.`
- All other ports must be blocked.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-to-backend-8080

spec:
  podSelector:
    matchLabels:
      app: backend                # ✅ Policy applies only to Backend Pods

  policyTypes:
  - Ingress                       # ✅ Control only Ingress Incoming Traffic

  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend           # ✅ Allow frontend pod only

    ports:
    - protocol: TCP
      port: 8080                  # ✅ Allow TCP/8080 with frontend pod only
```

Result Table

Source	Port	Status
Frontend	8080	✅ Allowed
Frontend	80	❌ Denied
Other Pods	8080	❌ Denied

- 🛡️ This NetworkPolicy allows ingress traffic to backend Pods only on TCP port 8080 with frontend pod only. All other ports are denied...

✅ Scenario 4: Allow Traffic from Specific Namespace

- 🎯 Requirement
  - 🛡️ Allow traffic only from Pods running in the `dev namespace`, Deny traffic from all other namespaces.

- Label the namespaces:
  - `kubectl label ns dev env=dev`
  - `kubectl label ns prod env=prod`
- Verify:
  - `kubectl get ns --show-labels`

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dev

spec:
 podSelector: {}                  #✅This policy applies to all Pods in this namespace

  policyTypes:
 - Ingress                        #✅Control only Incoming Traffic

  ingress:
  - from:
      - namespaceSelector:
          matchLabels:
            env: dev              #✅Allow Only dev Namespace
```

Result Table

Source Namespace	Label	Access
dev	env=dev	✅ Allowed
prod	env=prod	❌ Denied
test	env=test	❌ Denied
default	No label	❌ Denied

- 👉 This NetworkPolicy allows ingress traffic only from namespaces labeled env=dev.
- ❌Traffic from namespaces such as `prod`, `test`, or `default` is denied.

✅ Scenario 5: Allow Database Access Only from Backend

- 🎯 Requirement
  - Only the `Backend Pod` should be able to access the `MySQL Database` on `port 3306.`

- All other Pods must be denied.
- Architecture :

```
  Frontend
(app=frontend)
        |
        ▼
  Backend
(app=backend)
        |
TCP 3306
        ▼
Database
  (app=mysql)

Frontend ——► Database ❌ Denied
Other Pods —► Database ❌ Denied
```

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy

spec:
  podSelector:
    matchLabels:
      app: mysql                  #✅ This policy applies only to Database Pods (app=mysq

  policyTypes:
  - Ingress                       #✅ Controls traffic entering the database Pods (Ingre

  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: backend            #✅ Allow Only Backend Pods (Only Pods labeled app: ba

    ports:
    - port: 3306                  #✅ Only MySQL traffic is allowed (Only MySQL Port: 33
      protocol: TCP
```

Result Table

Source Pod	Port	DB Access
Backend	3306	✅ Allowed

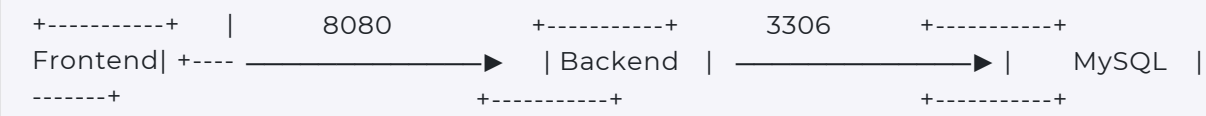
Source Pod	Port	DB Access
Frontend	3306	❌ Denied
Prometheus	3306	❌ Denied
Any Other Pod	3306	❌ Denied
Backend	80	❌ Denied

- 🛡️ This policy restricts both `source` and `port`
  - `Source = app=backend`
  - `Port = 3306`
- Both conditions must match., Backend + 3306 ✅ Allowed
- 👉 This NetworkPolicy allows only backend Pods (`app=backend`) to access MySQL Pods (`app=mysql`) on TCP port `3306`, while all other Pods and ports are `denied.`

✅ Scenario 6: 🏗️ Production 3-Tier App (Frontend → Backend → Database only)

- 🎯 Combined Ingress + Egress NetworkPolicy
- Instead of creating separate policies, we can create two NetworkPolicies in a `single YAML file:`
- Goal:
  - 🛡️ Frontend ➜ Backend = Allowed on Port 8080
  - 🛡️ Backend ➜ MySQL = Allowed on Port 3306
  - ❌ Everything else = Blocked

Architecture Diagram




```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:
    matchLabels:
```

```
      app: backend                #✅ Policy applies only to Backend Pods

  policyTypes:
  - Ingress
  - Egress

  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend           # ✅ (Allow:Frontend ---> Backend)
    ports:
    - protocol: TCP
    port: 8080                    # ✅ Frontend can access Backend only on (TCP 8080 Al

  egress:
  - to:
    - podSelector:
        matchLabels:
          app: mysql              # ✅ Backend can connect to MySQL only on TCP 3306
    ports:
    - protocol: TCP
      port: 3306

---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mysql-policy
spec:
  podSelector:
    matchLabels:
      app: mysql                  # ✅ Apply policy only to app=mysql.

  policyTypes:
  - Ingress

  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: backend            # ✅ Only Backend Pods can access MySQL (Backend → My
    ports:
    - protocol: TCP
      port: 3306
```

What is Allowed?

Source	Destination	Port	Status
Frontend	Backend	8080	✅ Allowed
Backend	MySQL	3306	✅ Allowed

- 👉 This combined NetworkPolicy allows only `Frontend Pods to access Backend` on port 8080 and allows `BackendPodstocommunicateonly with Database (MySQL) on port 3306`.
- ❌ All other ingress and egress traffic is denied.

Q: Backend accesses MySQL and MySQL returns data. Is Ingress alone sufficient?

- 🛡️ Yes. Kubernetes NetworkPolicies are `stateful.`
- ✅ If the MySQL Pod `allows Ingress from` the Backend Pod on port 3306, the response traffic automatically flows back through the established connection.
- 👉 An Egress policy is only required when MySQL initiates a `separate outbound connection` to another service. 🚀

Q: Why do we need both Backend Egress Policy and MySQL Ingress Policy?

- 👉 NetworkPolicies are directional means that Ingress and Egress traffic are controlled independently.
  - 🚦 Backend Egress controls where Backend can send traffic.
  - 🚦 MySQL Ingress controls who can access MySQL.
  - 🛡️ Formaximum security, use both :
    - ✅ `Backend Egress → Allow Backend → MySQL`
    - ✅ `MySQL Ingress → Allow only Backend → MySQL`
    - 🔐 This follows the `least privilege security model in Kubernetes`.